



Université de Rouen Normandie

SSL Checker

Spécification technique des besoins

Spécification Technique Des Besoins

Étudiants :

Mehdi LASMI

Marzouk MONIR

Malik MAHFOUDI

Karina AIT-ABDELKAOUI

Naby TRAORE SAVANE

Rayan BOUGHANEM

Hamza RAHMOUNI

Amr ELGOHARY

Client :

Bruno MACADRÉ

Table des matières

1 Terminologie et sigles utilisés **2**

2 Introduction **4**

2.1 Contexte du projet 4

2.2 Objectif général 4

2.3 Objectifs spécifiques 4

2.3.1 Core de l'Analyse (Tests Indispensables) 4

2.3.2 Interface et Accessibilité 4

2.3.3 Tests Avancés et Améliorations 5

3 Périmètre fonctionnel **5**

3.1 Périmètre Inclus 5

3.1.1 Interface Utilisateur et Flux d'Analyse 5

3.1.2 Tests de Qualité SSL/TLS 5

3.1.3 Rapport et Notation 6

3.2 Périmètre Exculs 6

4 Acteurs **6**

5 Cas D'utilisation **6**

6 Diagrammes **7**

6.1 Diagramme de séquence générale 7

6.2 Diagramme de cas d'utilisation générale 8

7 Exigences fonctionnelles **8**

8 Exigences non-fonctionnelles **10**

9 Critères de validation et tests **11**

9.1 Validation fonctionnelle 11

9.1.1 Méthodes / cas de test 11

9.2 Validation de sécurité 11

9.2.1 Objectifs 11

9.2.2 Méthodes / cas de test 11

1 Terminologie et sigles utilisés

— **CA (Certificate Authority)** : Autorité de certification chargée de délivrer et de vérifier les certificats numériques.
— **Cipher Suite** : Ensemble d'algorithmes utilisés pour sécuriser une connexion TLS/SSL (chiffrement, échange de clés, intégrité, etc.).
— **CRL (Certificate Revocation List)** : Liste publiée par une CA contenant les identifiants des certificats révoqués avant leur date d'expiration.
— **DNS (Domain Name System)** : Système de résolution qui traduit les noms de domaine en adresses IP.
— **EKU (Extended Key Usage)** : Extension X.509 définissant les finalités spécifiques pour lesquelles la clé publique certifiée peut être utilisée (ex : authentification serveur).
— **FQDN (Fully Qualified Domain Name)** : Nom de domaine complet identifiant précisément une machine sur Internet (ex. : `www.exemple.com`).
— **Heartbleed** : Faille de sécurité critique dans certaines versions d'OpenSSL permettant la fuite de données sensibles en mémoire.
— **HTTP/HTTPS (HyperText Transfer Protocol / Secure)** : Protocole de communication utilisé sur le Web ; HTTPS est la version sécurisée via TLS/SSL.
— **OCSP (Online Certificate Status Protocol)** : Protocole internet permettant de vérifier l'état de révocation d'un certificat numérique en temps réel.
— **POODLE (Padding Oracle On Downgraded Legacy Encryption)** : Vulnérabilité affectant SSL 3.0 permettant le déchiffrement partiel de données.
— **RSA (Rivest-Shamir-Adleman)** : Algorithme de cryptographie asymétrique largement utilisé pour l'échange de clés et la signature numérique.
— **SAN (Subject Alternative Name)** : Extension permettant d'associer plusieurs noms d'hôtes (domaines, sous-domaines, IP) à un seul certificat SSL/TLS.
— **SHA (Secure Hash Algorithm)** : Famille de fonctions de hachage (ex : SHA-256) utilisée pour garantir l'intégrité des données et des signatures numériques.
— **SQLI (SQL Injection)** : Type d'attaque consistant à injecter du code SQL malveillant dans une requête afin d'altérer ou d'extraire des données.
— **SSL (Secure Sockets Layer)** : Ancien protocole de sécurisation des communications sur Internet, prédécesseur obsolète du TLS.
— **SSL Labs** : Plateforme d'analyse en ligne permettant d'évaluer la configuration TLS/SSL d'un serveur web.
— **STB (Spécification Technique des Besoins)** : Document décrivant de manière détaillée les exigences techniques d'un projet.
— **Sweet32** : Attaque exploitant les faiblesses des chiffrements par blocs de 64 bits (comme 3DES) dans certains modes TLS.

— **TLS (Transport Layer Security)** : Protocole de sécurité cryptographique succédant au SSL, assurant la confidentialité et l'intégrité des données.
— **XSS (Cross-Site Scripting)** : Vulnérabilité web permettant l'injection de scripts malveillants dans des pages consultées par d'autres utilisateurs.

2 Introduction

2.1 Contexte du projet

Le projet SSLChecker est développé dans un contexte où la généralisation du protocole HTTPS pour les sites web et services en ligne pose une question fondamentale : l'ajout d'un certificat est-il suffisant pour garantir la robustesse d'un serveur web face aux menaces actuelles ?

Ce projet vise à appliquer les principes de sécurité pour évaluer de manière critique la configuration des serveurs. L'outil est principalement destiné à fonctionner en intranet mais est conçu pour pouvoir fonctionner sur Internet si la capacité technique le permet.

2.2 Objectif général

L'objectif général du projet est de concevoir et de réaliser un outil web permettant de tester et d'évaluer la qualité SSL/TLS (Sécurité) d'un serveur web cible.

Cet outil doit s'inspirer du service SSL Labs de Qualys pour s'en rapprocher au maximum dans son fonctionnement et des tests effectués. Il doit fournir une analyse complète des aspects de sécurité et attribuer une note globale au serveur

2.3 Objectifs spécifiques

Les objectifs spécifiques décrivent les fonctionnalités critiques que l'outil doit implémenter pour atteindre l'objectif général.

2.3.1 Core de l'Analyse (Tests Indispensables)

- Vérification de la Configuration du Serveur : Déterminer les protocoles (SSL/TLS), les suites de chiffrement ainsi que les algorithmes d'échange de clés supportés par le serveur.
- Analyse du Certificat : Remonter et analyser les informations clés du certificat ainsi que la chaîne de certification complète
- Notation Personnalisée : Développer un système de notation propre pour évaluer la sécurité selon nos critères, en justifiant clairement les pondérations attribuées.

2.3.2 Interface et Accessibilité

- Fournir une interface web simple et agréable , permettant la saisie d'une adresse FQDN avec port optionnel.
- Afficher une page de résultat complète avec l'ensemble des tests effectués.
- L'application finale doit être sécurisée en HTTPS.

2.3.3 Tests Avancés et Améliorations

— Résistance aux Attaques : Intégrer la simulation des attaques connues (ex : Heartbleed, POODLE) pour évaluer la robustesse du serveur, ce qui est particulièrement utile pour les serveurs plus anciens.
— Améliorations Optionnelles : Intégrer des fonctionnalités avancées comme le fingerprinting pour identifier le moteur de service web (Apache, Nginx, etc.) et proposer des améliorations de configuration basées sur les résultats.

3 Périmètre fonctionnel

3.1 Périmètre Inclus

3.1.1 Interface Utilisateur et Flux d'Analyse

— **Saisie Cible** : Permettre à l'utilisateur de saisir une adresse FQDN ainsi qu'un port optionnel (par défaut 443) à analyser.
— **Contrôle de l'Analyse** : Déclencher le Moteur d'Analyse TLS qui teste l'exhaustivité des protocoles et des suites de chiffrement supportés.
— **Visualisation des Résultats** : Assurer l'affichage immédiat du rapport de sécurité complet (matrice de compatibilité et note globale) une fois la compilation des données terminée.

3.1.2 Tests de Qualité SSL/TLS

— **Analyse du Certificat** : Vérification de la validité générale (dates, nom de domaine),vérification de la taille de la clé et vérification de la chaîne de certification (complète, correcte, et ancrée à une CA de confiance).
— **Protocoles Supportés** : Détection des versions supportées.
— **Suites de Chiffrement** : Identification de toutes les suites supportées et de l'ordre de préférence du serveur.
— **Échange de clés** : Identification de la liste des algorithmes d'échange de clés supportés par le serveur, ainsi que de la taille de clé associée pour chacun.
— **Tests de Résistance aux Attaques** : Vérification de la configuration pour des vulnérabilités célèbres.

3.1.3 Rapport et Notation

— **Affichage Détaillé** : Présenter la matrice de compatibilité Protocole x Ciphers de manière lisible (tableaux, codes couleurs).
— **Notation Globale** : Attribuer une note de qualité basée sur les différents axes de teste.

3.2 Périmètre Exculs

— **Gestion des Utilisateurs** :Le projet sera conçu sans gestion d'authentification, de profils utilisateurs ou de sauvegarde des historiques d'analyse.
— **Tests Applicatifs** : L'outil ne testera pas les vulnérabilités applicatives (SQLi, XSS, etc.),car il se concentre uniquement sur la sécurité de la couche de transport (TLS/HTTPS).

4 Acteurs

Acteur	Rôle et Responsabilités	Interactions Spécifiques
1. Utilisateur	Entité humaine qui initie l'intégralité du cycle d'analyse. Fournit les données d'entrée (FQDN, Port]) et consomme les livrables (Rapport, Note).	Interagit uniquement avec l'Interface Web (Frontend) via HTTP/HTTPS pour envoyer les requêtes et recevoir le rapport.

TABLE 1 – Acteurs, rôles et interactions

5 Cas D'utilisation

Nom : Lancer une analyse

Acteurs concernés : Utilisateur

Description : L'utilisateur a la capacité d'envoyer un FDQN et éventuellement un port.

Préconditions :
— L'utilisateur doit saisir une entrée au bon format

Évènement déclenchant : L'utilisateur clique sur le bouton « Lancer analyse ».

Conditions d'arrêt :

— Analyse terminée avec succès

— Entrée non reconnue

Enchaînement nominal :

1. Traitement de l'entrée :
   — FQDN → résolution DNS
2. Vérification de la chaîne des certificats
3. Vérification des protocoles supportés
4. Vérification des suites cryptographiques supportées
5. Tests de vulnérabilités (POODLE, Sweet32, HeartBleed)
6. Vérification de la compatibilité avec les OS
7. Scan des ports de la cible
8. Calcul d'un score global
9. Génération d'un compte rendu

Flots d'exceptions : Si une erreur se produit, le système affiche un message d'erreur.

Table : Description textuelle pour le cas d'utilisation — Lancer une analyse

6 Diagrammes

6.1 Diagramme de séquence générale

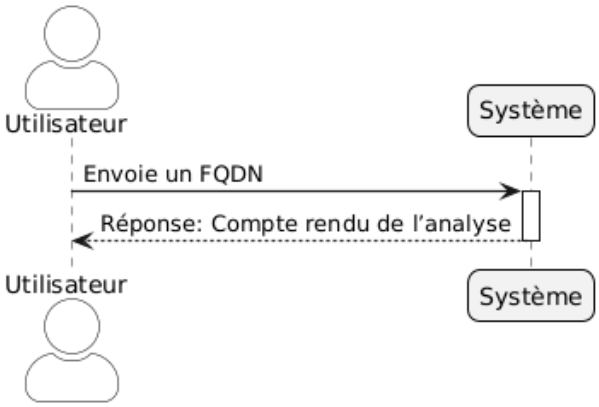


Diagramme de séquence générale

6.2 Diagramme de cas d'utilisation générale

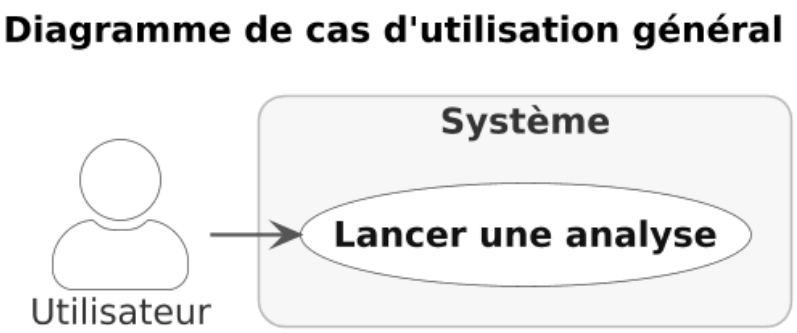


Diagramme de cas d'utilisation génerale

7 Exigences fonctionnelles

ID	Catégorie	Description de la règle	Priorité
EF-01	Entrée utilisateur	Le système doit accepter un FQDN et port valide	Haute
EF-02	Entrée utilisateur	Le système doit obtenir l'adresse IP cible avant toute analyse	Haute
EF-03	Certificat SSL/TLS	Le système doit vérifier la présence du certificat SSL/TLS	Haute
EF-04	Certificat SSL/TLS	Le système doit valider la date d'expiration du certificat	Haute
EF-05	Certificat SSL/TLS	Le système doit vérifier l'émetteur du certificat (autorité de certification)	Haute
EF-06	Certificat SSL/TLS	Le système doit analyser la validité de la chaîne de certificats	Haute
EF-07	Protocoles	Le système doit vérifier les versions de protocoles SSL/TLS supportées par le serveur	Haute
EF-08	Protocoles	Le système doit déterminer si la cible autorise les protocoles obsolètes (SSLv2, SSLv3, TLSv1.0, TLSv1.1)	Haute
EF-9	Cipher Suites	Le système doit lister toutes les cipher suites supportées par le serveur	Moyenne
EF-10	Cipher Suites	Le système doit identifier les cipher suites obsolètes ou vulnérables	Haute

Suite à la page suivante...

N° Règle	Catégorie	Description de la règle	Priorité
EF-11	Vulnérabilités	Le système doit tester la vulnérabilité à l'attaque Heartbleed	Moyenne
EF-12	Vulnérabilités	Le système doit tester la vulnérabilité à l'attaque configuration forcée et POODLE	Moyenne
EF-13	Compatibilité	Le système doit lister tous les systèmes d'exploitation compatibles avec la configuration SSL/TLS du serveur	Basse
EF-14	Scan Réseau	Le système doit effectuer un scan des ports principaux (80, 443, 21, 22, 25, etc.)	Basse
EF-15	Scan Réseau	Le système doit déterminer l'état de chaque port scanné	Basse
EF-16	Notation	Le système doit attribuer une note pour la qualité du certificat	Haute
EF-17	Notation	Le système doit attribuer une note pour la configuration des protocoles	Haute
EF-18	Notation	Le système doit attribuer une note pour la configuration des cipher suites	Haute
EF-19	Notation	Le système doit calculer une note globale à partir des notes certificat, protocoles et cipher suites	Haute
EF-20	Rapport	Le système doit générer un rapport détaillé contenant tous les résultats d'analyse	Haute
EF-21	Cipher suites	le systéme doit classer les suites de chiffrement selon la préférence du serveur	Moyenne
EF-22	Certificat SSL/TLS	Le système doit vérifier les extensions du certificat, notamment la présence du domaine dans le SAN (Subject Alternative Name) et l'usage de la clé (EKU).	Haute
EF-23	Certificat SSL/TLS	Le système doit vérifier le statut de révocation du certificat auprès de l'autorité (via OCSP ou CRL) pour s'assurer qu'il est toujours valide.	Haute
EF-24	Certificat SSL/TLS	Le système doit analyser la robustesse de la clé publique du certificat (type et taille, ex : RSA 2048) et de l'algorithme de signature (ex : SHA-256).	Haute

Suite à la page suivante...

N° Règle	Catégorie	Description de la règle	Priorité
EF-25	Échange de clés	Le système doit déterminer la liste des algorithmes d'échange de clés supportés par le serveur.	Haute
EF-26	Echange de cles	Le système doit déterminer la taille de la clé d'échange associée à chaque algorithme d'échange de clés supporté par le serveur (bits ou nom de courbe).	Haute

8 Exigences non-fonctionnelles

ID	Exigence Non Fonctionnelle	Description	Critère / Priorité
ENF-01	Performance	Temps d'analyse raisonnable et interface réactive	$<$ 5 minutes / Haute
ENF-02	Disponibilité	Accessibilité de l'outil en intranet	$>$ 99% / Moyenne
ENF-03	Utilisabilité	Interface intuitive et claire, inspirée de SSL Labs	Prise en main immédiate / Haute
ENF-04	Fiabilité	Gestion robuste des erreurs (cible injoignable)	Aucun crash / Haute
ENF-05	Sécurité	Protection contre les injections et abus	Validation des entrées / Haute
ENF-06	Précision	Exactitude des résultats et des détections	Sources à jour / Essentielle
ENF-07	Maintenabilité	Code structuré pour faciliter les ajouts	Architecture modulaire / Moyenne
ENF-08	Portabilité	Compatibilité avec environnements standards	Support Linux/Windows / Moyenne

Table 3 – Exigences Non Fonctionnelles

9 Critères de validation et tests

9.1 Validation fonctionnelle

Vérification que l'application réalise toutes les fonctionnalités demandées par le cahier des charges :

- Elle effectue bien les tests sur les chiffrements, le certificat et les protocoles utilisés (ex. TLS 1.2/1.3).
- Elle peut résoudre un nom de domaine (FQDN) pour lancer les tests.
- L'application affiche correctement les résultats (rapport lisible, valeurs précises, statuts d'erreur si nécessaire).

9.1.1 Méthodes / cas de test

- Faire plusieurs tests sur plusieurs sites réels (ex. domaine avec certificat valide, domaine avec certificat expiré, domaine avec suites faibles).
- Tester les formats d'entrée invalides : nom de domaine mal formé, caractères spéciaux, très longues chaînes, FQDN et port invalide. L'application doit valider l'entrée et montrer un message d'erreur clair.
- Vérifier la présentation des résultats : lisibilité, ordre logique, légendes, absence d'informations contradictoires.
- Vérifier les messages d'erreur et d'alerte (doivent être précis et utiles).

9.2 Validation de sécurité

Vérifier que l'application est protégée contre les vulnérabilités et que les utilisateurs sont protégés lorsqu'ils utilisent notre application.

9.2.1 Objectifs

- Empêcher les intrusions et les attaques courantes sur les champs.

9.2.2 Méthodes / cas de test

- Utiliser HTTPS/TLS.
- Désactiver les méthodes HTTP inutiles.
- Faire de la programmation défensive côté serveur.

Bruno Macadre

